

Hexabid Architecture

Table of Contents

1. Cel dokumentu	1
2. Założenia	1
3. Widok 1: System Context	2
4. Widok 2: Container	2
5. Widok 3: Component	3
6. Główne scenariusze	4
6.1. Tworzenie aukcji	4
6.2. Licytacja w czasie rzeczywistym	4
6.3. Zamknięcie wygasłych aukcji	4
7. Decyzje architektoniczne	4
8. Renderowanie dokumentu	5

1. Cel dokumentu

Ten dokument opisuje architekturę systemu **Hexabid** w modelu C4. Zakres obejmuje:

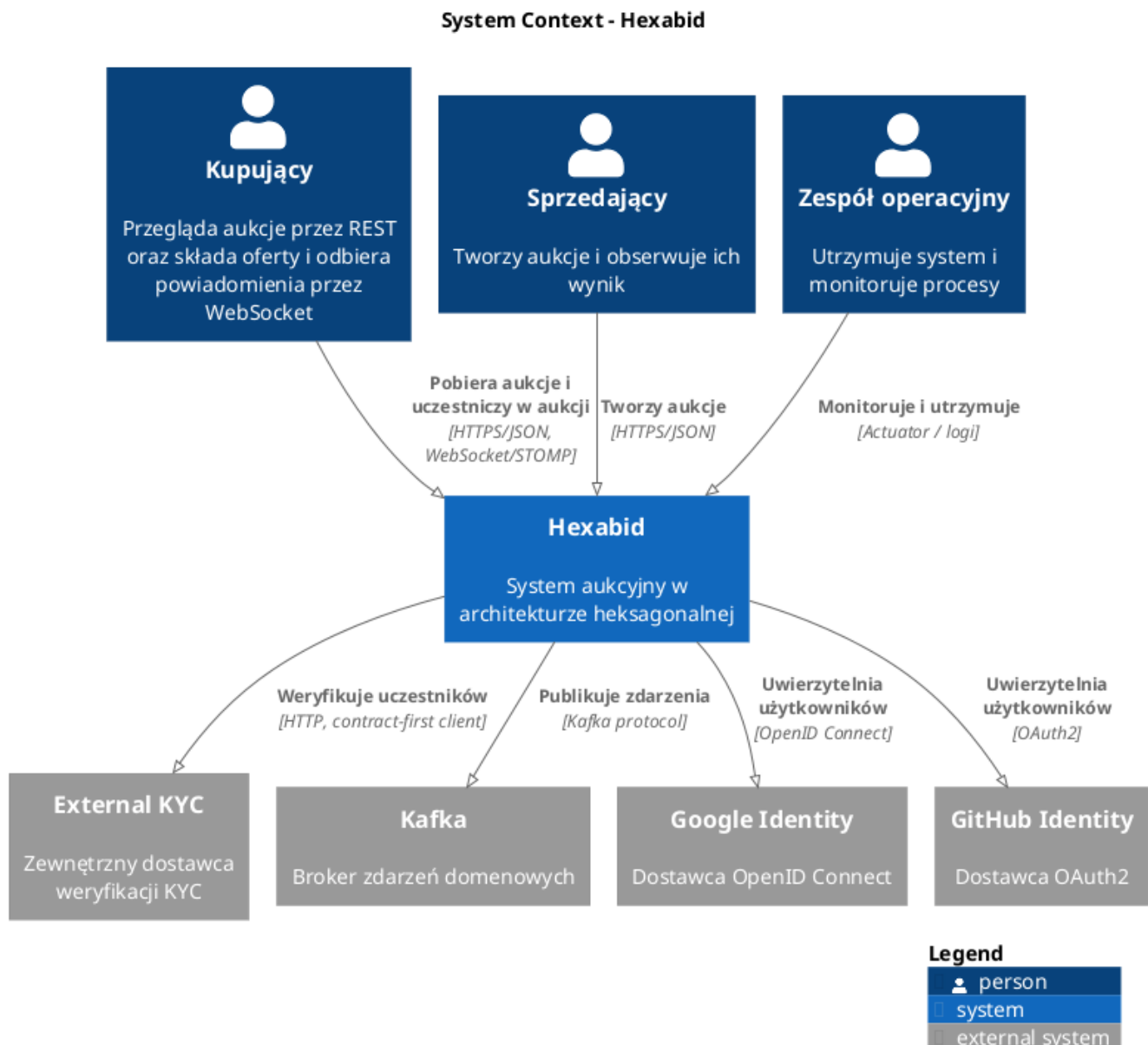
- kontekst systemowy
- kontenery aplikacyjne i integracyjne
- główne komponenty wewnątrz rdzenia i adapterów

Dokument jest zapisany w AsciiDoc, a diagramy są osadzone jako PlantUML z biblioteką **C4-PlantUML**.

2. Założenia

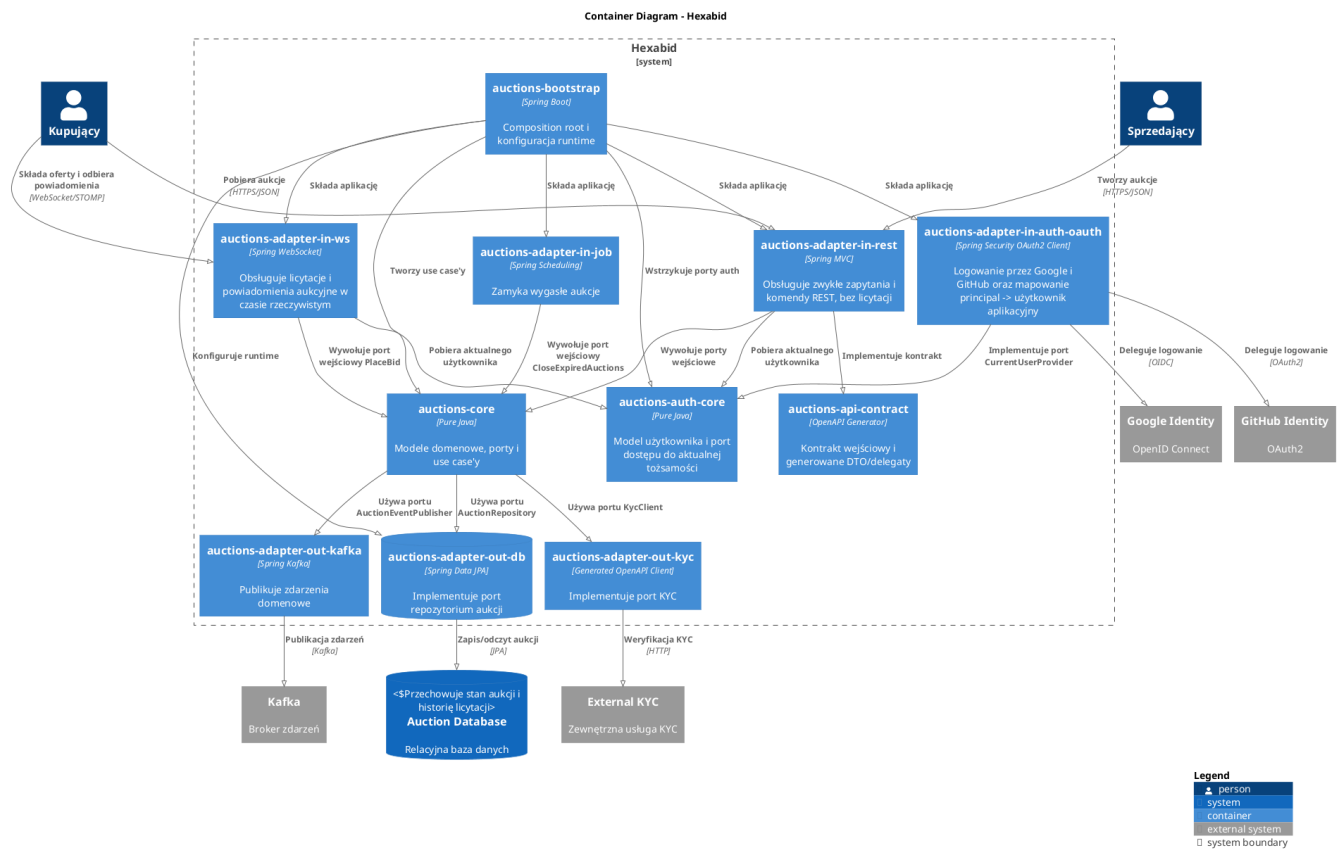
- system jest budowany jako wielomodułowy projekt Maven
- domena aukcyjna pozostaje niezależna od Springa i JPA
- wspólny archetyp **Party** znajduje się w rdzeniu i reprezentuje uczestnika biznesowego niezależnie od mechanizmu logowania
- uwierzytelnianie jest adapterem wejściowym, a tożsamość użytkownika jest dostarczana do aplikacji przez port
- kontrakty HTTP są definiowane przed implementacją
- adaptory wejściowe i wyjściowe komunikują się z **actions-core** wyłącznie przez porty

3. Widok 1: System Context

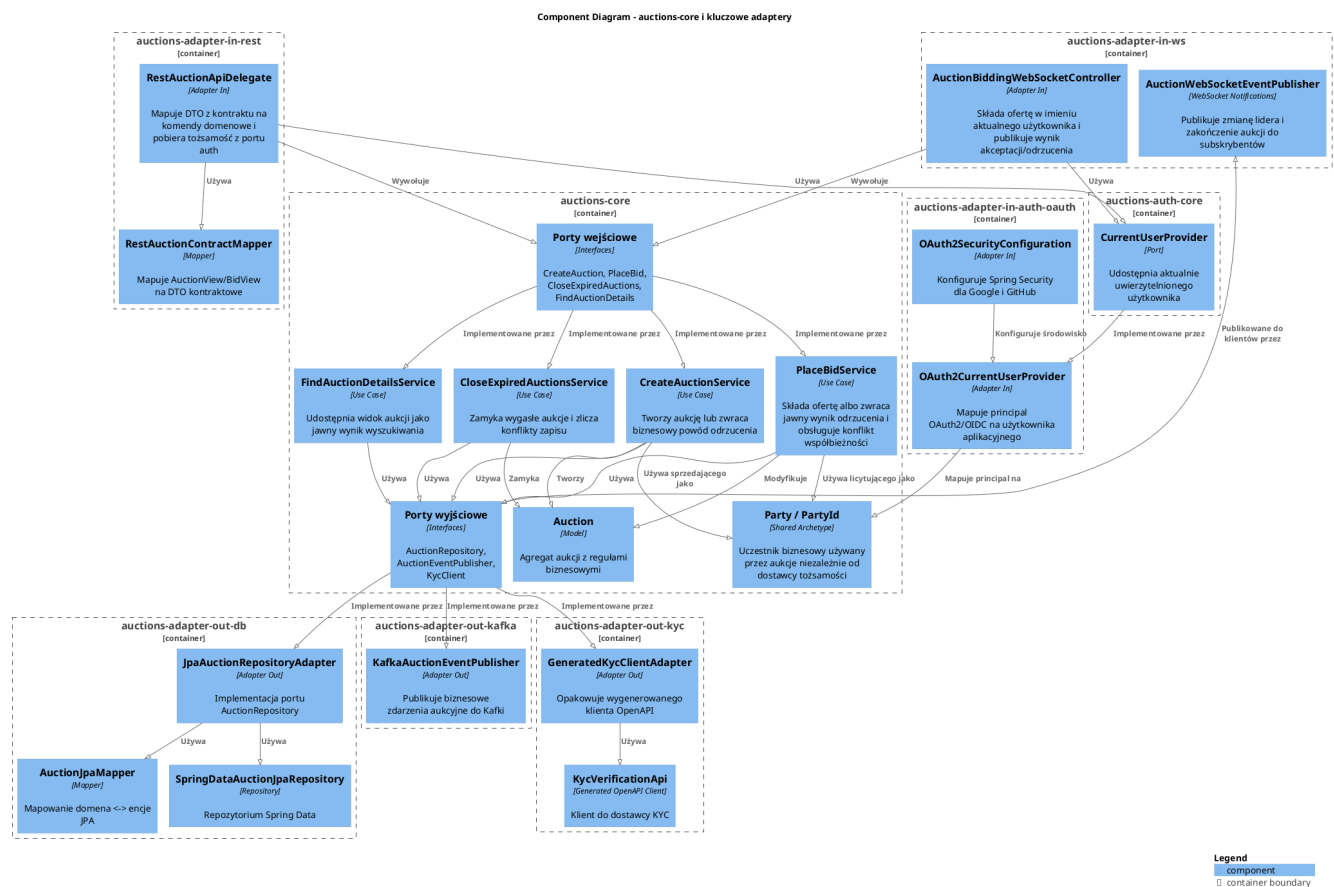


Na poziomie **System Context** nie pokazuję **Job** jako osobnego inicjatora, ponieważ w tej aplikacji scheduler jest elementem wewnętrznym systemu, a nie zewnętrznym aktorem lub osobnym systemem. Dlatego pojawia się dopiero na poziomie **Container**.

4. Widok 2: Container



5. Widok 3: Component



6. Główne scenariusze

6.1. Tworzenie aukcji

1. Sprzedający wywołuje REST `POST /api/auctions`.
2. `auctions-adapter-in-rest` pobiera aktualną tożsamość z `CurrentUserProvider` i mapuje request na `CreateAuctionCommand`.
3. `CreateAuctionService` sprawdza KYC przez port `KycClient`.
4. `CreateAuctionService` zwraca jawny wynik utworzenia albo biznesowy powód odrzucenia.
5. `AuctionRepository` zapisuje nową aukcję.
6. REST zwraca widok aukcji zgodny z kontraktem OpenAPI.

6.2. Licytacja w czasie rzeczywistym

1. Kupujący wysyła ofertę przez WebSocket.
2. Adapter WebSocket pobiera aktualną tożsamość użytkownika z portu auth.
3. `PlaceBidService` wywołuje agregat i zwraca `BidAccepted` albo `BidRejected`.
4. `Auction` waliduje reguły licytacji i aktualizuje historię ofert.
5. `AuctionRepository` zapisuje nowy stan agregatu z optimistic locking.
6. `AuctionEventPublisher` publikuje zdarzenie zmiany lidera do Kafki i do kanału WebSocket.
7. W przypadku konfliktu współbieżności use case zwraca wynik biznesowy `CONCURRENT_MODIFICATION`, a system nie traci stabilności.

6.3. Zamknięcie wygasłych aukcji

1. Scheduler uruchamia `CloseExpiredAuctionsUseCase`.
2. Core pobiera wygasłe aukcje z `AuctionRepository`.
3. `Auction` zmienia status na `CLOSED` i rozstrzyga, czy aukcja została wygrana czy zamknięta bez zwycięzcy.
4. W razie konfliktu optimistic lock aukcja jest pomijana, a use case zlicza konflikt zamiast zatrzymać cały job.
5. Adaptery publikują biznesowe zdarzenie końca aukcji do Kafki i WebSocket.

7. Decyzje architektoniczne

- `auctions-core` nie zna Springa, JPA, Kafki ani HTTP.
- `auctions-auth-core` izoluje model użytkownika i port dostępu do aktualnej tożsamości od Spring Security.
- `auctions-bootstrap` pełni rolę composition root i tworzy use case'y jako beany.

- komunikacja REST i KYC jest contract-first, dzięki generatorom OpenAPI.
- uwierzytelnianie odbywa się przez OAuth2/OpenID Connect z dostawcami Google i GitHub.
- licytacja i powiadomienia aukcyjne są prowadzone przez WebSocket, a REST służy do zwykłych operacji i odczytów.
- zdarzenia domenowe opisują konkretny fakt biznesowy, a nie ogólne "coś się zakończyło".
- use case'y zwracają jawne wyniki biznesowe zamiast przeciekać techniczne wyjątki do adapterów.
- zapis aukcji wykorzystuje optimistic locking; konflikty są obsługiwane stabilnie i jawnie.
- pakiety domenowe są zorientowane na język biznesowy, a nie na technologię.

8. Renderowanie dokumentu

HTML:

```
asciidoctor \
  -r asciidoctor-diagram \
  -r asciidoctor-diagram/plantuml/classpath \
  doc/architecture-c4.adoc
```

PDF:

```
asciidoctor-pdf \
  -r asciidoctor-diagram \
  -r asciidoctor-diagram/plantuml/classpath \
  -a pdf-theme=doc/pdf-theme.yml \
  -o doc/architecture-c4.pdf \
  doc/architecture-c4.adoc
```

Uwaga:

- PDF generowany z IDE może pokazywać placeholder zamiast diagramów, jeśli plugin AsciiDoc nie ma aktywnego backendu diagramów.
- Renderowanie przez CLI z powyższymi **require** osadza diagramy poprawnie.
- Motyw **doc/pdf-theme.yml** wymusza font **Noto Sans**, żeby PDF poprawnie wyświetlał polskie znaki.
- Jeżeli środowisko blokuje zdalne **!includeurl**, można zmirrorować bibliotekę **C4-PlantUML** lokalnie i podmienić dyrektywy **!includeurl** na **!include**.